

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил:

А. В. Шевчук

студент группы ПО-13

Проверил:

А. Д. Кулик

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Тема: Применение порождающих, структурных и поведенческих паттернов проектирования

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

1. Цель работы

Цель лабораторной работы — освоить применение трёх основных групп паттернов проектирования (порождающие, структурные и поведенческие) при реализации программных систем на языке Python. В ходе выполнения работы необходимо выбрать по одному заданию из каждой группы, определить подходящий паттерн, обосновать выбор и реализовать программный фрагмент.

2. Задание по лабораторной работе

2.1. Порождающий паттерн (вариант 7)

Тема: Преподаватель. Класс должен обеспечивать одновременное взаимодействие с несколькими объектами класса Студент. Основные функции преподавателя – Проверить Лабораторную Работу, Провести Консультацию, Принять Экзамен, Выставить Отметку, Провести Лекцию.

```
"""Lab 3, task 1, variant 7.
```

Порождающий паттерн: Factory Method.

Преподаватель работает с несколькими студентами, которые создаются через фабрики.

```
"""
```

```
from abc import ABC, abstractmethod
```

```
# pylint: disable=too-few-public-methods
```

```
class Student(ABC):
```

```
    """Абстрактный студент."""
```

```
    def __init__(self, name: str) -> None:
```

```
        if not name.strip():
```

```
            raise ValueError("Имя студента не может быть пустым.")
```

```
        self.name = name
```

```
    @abstractmethod
```

```
    def info(self) -> str:
```

```
        """Вернуть описание студента."""
```

```
# pylint: disable=too-few-public-methods
```

```
class BachelorStudent(Student):
```

```
    """Студент-бакалавр."""
```

```
    def info(self) -> str:
```

```
        return f"Студент-бакалавр: {self.name}"
```

```

# pylint: disable=too-few-public-methods
class MasterStudent(Student):
    """Студент-магистр."""

    def info(self) -> str:
        return f'Студент-магистр: {self.name}'

# pylint: disable=too-few-public-methods
class StudentFactory(ABC):
    """Абстрактная фабрика студентов."""

    @abstractmethod
    def create_student(self, name: str) -> Student:
        """Создать студента."""

# pylint: disable=too-few-public-methods
class BachelorFactory(StudentFactory):
    """Фабрика бакалавров."""

    def create_student(self, name: str) -> Student:
        return BachelorStudent(name)

# pylint: disable=too-few-public-methods
class MasterFactory(StudentFactory):
    """Фабрика магистров."""

    def create_student(self, name: str) -> Student:
        return MasterStudent(name)

class Teacher:
    """Преподаватель, работающий с несколькими студентами."""

    def __init__(self, name: str) -> None:
        if not name.strip():
            raise ValueError("Имя преподавателя не может быть пустым.")
        self.name = name
        self.students: list[Student] = []

    def add_student(self, student: Student) -> None:
        """Добавить студента."""
        self.students.append(student)

    def conduct_lecture(self) -> None:
        """Провести лекцию."""
        print(f'{self.name} проводит лекцию.')
        for student in self.students:
            print(student.info())

```

```

def conduct_consultation(self) -> None:
    """Провести консультацию."""
    print(f'{self.name} проводит консультацию.')

def check_laboratory_work(self) -> None:
    """Проверить лабораторную работу."""
    print(f'{self.name} проверяет лабораторную работу.')

def accept_exam(self) -> None:
    """Принять экзамен."""
    print(f'{self.name} принимает экзамен.')

def assign_grade(self) -> None:
    """Выставить отметки."""
    print(f'{self.name} выставляет отметки.')

def main() -> None:
    """Демонстрация работы паттерна."""
    bachelor_factory = BachelorFactory()
    master_factory = MasterFactory()

    teacher = Teacher("Крощенко А.А.")
    teacher.add_student(bachelor_factory.create_student("Иван"))
    teacher.add_student(bachelor_factory.create_student("Алина"))
    teacher.add_student(master_factory.create_student("Дмитрий"))

    teacher.conduct_lecture()
    teacher.conduct_consultation()
    teacher.check_laboratory_work()
    teacher.accept_exam()
    teacher.assign_grade()

if __name__ == "__main__":
    main()

```

Ввод: нет **Вывод:** Пример:

```

Крощенко А.А. проводит лекцию.
Студент-бакалавр: Иван
Студент-бакалавр: Алина
Студент-магистр: Дмитрий
Крощенко А.А. проводит консультацию.
Крощенко А.А. проверяет лабораторную работу.
Крощенко А.А. принимает экзамен.
Крощенко А.А. выставляет отметки.

```

2.2. Структурный паттерн (вариант 7)

Тема: ДУ автомобиля. Реализовать иерархию автомобилей для конкретных производителей и иерархию средств дистанционного управления. Автомобили должны иметь присущие им атрибуты и функции. ДУ имеет три

основные функции – удаленная активация сигнализации, удаленное открытие/закрытие дверей и удаленный запуск двигателя. Эти функции должны отличаться по своим характеристикам для различных устройств ДУ.

""Lab 3, task 2, variant 7.

Структурный паттерн: Bridge.

Автомобили и пульта дистанционного управления развиваются независимо друг от друга.

""

```
from abc import ABC, abstractmethod
```

```
class Car(ABC):
```

```
    """Абстрактный автомобиль."""
```

```
    def __init__(self, model: str) -> None:
```

```
        if not model.strip():
```

```
            raise ValueError("Модель не может быть пустой.")
```

```
        self.model = model
```

```
        self.alarm = False
```

```
        self.locked = False
```

```
        self.engine = False
```

```
    @abstractmethod
```

```
    def brand(self) -> str:
```

```
        """Вернуть марку автомобиля."""
```

```
    def activate_alarm(self) -> None:
```

```
        """Включить сигнализацию."""
```

```
        self.alarm = True
```

```
        print(f'{self.brand()} {self.model}: сигнализация включена.")
```

```
    def deactivate_alarm(self) -> None:
```

```
        """Выключить сигнализацию."""
```

```
        self.alarm = False
```

```
        print(f'{self.brand()} {self.model}: сигнализация выключена.")
```

```
    def lock_doors(self) -> None:
```

```
        """Заблокировать двери."""
```

```
        self.locked = True
```

```
        print(f'{self.brand()} {self.model}: двери заблокированы.")
```

```
    def unlock_doors(self) -> None:
```

```
        """Разблокировать двери."""
```

```
        self.locked = False
```

```
        print(f'{self.brand()} {self.model}: двери разблокированы.")
```

```
    def start_engine(self) -> None:
```

```
        """Запустить двигатель."""
```

```
        self.engine = True
```

```
        print(f'{self.brand()} {self.model}: двигатель запущен.")
```

```
    def stop_engine(self) -> None:
```

```
        """Остановить двигатель."""
```

```
        self.engine = False
```

```

        print(f"{self.brand()} {self.model}: двигатель остановлен.")

# pylint: disable=too-few-public-methods
class BMW(Car):
    """Автомобиль BMW."""

    def brand(self) -> str:
        return "BMW"

# pylint: disable=too-few-public-methods
class Toyota(Car):
    """Автомобиль Toyota."""

    def brand(self) -> str:
        return "Toyota"

# pylint: disable=too-few-public-methods
class Audi(Car):
    """Автомобиль Audi."""

    def brand(self) -> str:
        return "Audi"

# pylint: disable=too-few-public-methods
class RemoteControl(ABC):
    """Абстрактный пульт управления."""

    def __init__(self, car: Car) -> None:
        self.car = car

    @abstractmethod
    def toggle_alarm(self) -> None:
        """Переключить сигнализацию."""

    @abstractmethod
    def toggle_doors(self) -> None:
        """Переключить двери."""

    @abstractmethod
    def toggle_engine(self) -> None:
        """Переключить двигатель."""

class StandardRemote(RemoteControl):
    """Обычный пульт."""

    def toggle_alarm(self) -> None:
        if self.car.alarm:
            self.car.deactivate_alarm()
        else:
            self.car.activate_alarm()

    def toggle_doors(self) -> None:
        if self.car.locked:
            self.car.unlock_doors()
        else:
            self.car.lock_doors()

```

```

def toggle_engine(self) -> None:
    if self.car.engine:
        self.car.stop_engine()
    else:
        self.car.start_engine()

class SecureRemote(RemoteControl):
    """Защищённый пульт."""

    def toggle_alarm(self) -> None:
        print("Проверка безопасности пройдена.")
        if self.car.alarm:
            self.car.deactivate_alarm()
        else:
            self.car.activate_alarm()

    def toggle_doors(self) -> None:
        print("Защищённое управление дверями.")
        if self.car.locked:
            self.car.unlock_doors()
        else:
            self.car.lock_doors()

    def toggle_engine(self) -> None:
        print("Защищённое управление двигателем.")
        if self.car.engine:
            self.car.stop_engine()
        else:
            self.car.start_engine()

def main() -> None:
    """Демонстрация работы паттерна."""
    bmw = BMW("X5")
    toyota = Toyota("Camry")
    audi = Audi("A6")

    standard_remote = StandardRemote(bmw)
    secure_remote = SecureRemote(toyota)
    another_standard_remote = StandardRemote(audi)

    print("=== BMW ===")
    standard_remote.toggle_doors()
    standard_remote.toggle_alarm()
    standard_remote.toggle_engine()

    print("\n=== Toyota ===")
    secure_remote.toggle_doors()
    secure_remote.toggle_alarm()
    secure_remote.toggle_engine()

    print("\n=== Audi ===")
    another_standard_remote.toggle_doors()
    another_standard_remote.toggle_engine()

if __name__ == "__main__":

```

main()

Ввод: нет. **Вывод:**

Пример вывода:

```
=== BMW ===  
BMW X5: двери заблокированы.  
BMW X5: сигнализация включена.  
BMW X5: двигатель запущен.
```

2.3. Поведенческий паттерн (вариант 7)

Тема: Проект «Пиццерия». Реализовать формирование заказ(а)ов, их отмену, а также повторный заказ с теми же самыми позициями.

"""Lab 3, task 3, variant 7.

Поведенческий паттерн: Command.
Пиццерия поддерживает создание заказа,
его отмену и повтор.
"""

```
from abc import ABC, abstractmethod
```

```
# pylint: disable=too-few-public-methods  
class Order:
```

```
    """Сущность заказа."""
```

```
    def __init__(self, order_id: int, items: list[str]) -> None:  
        self.order_id = order_id  
        self.items = items  
        self.status = "Создан"
```

```
    def info(self) -> str:  
        """Вернуть описание заказа."""  
        items_text = ", ".join(self.items)  
        return f"Заказ №{self.order_id}: {items_text}, статус={self.status}"
```

```
class Pizzeria:
```

```
    """Получатель команд."""
```

```
    def __init__(self) -> None:  
        self.orders: dict[int, Order] = {}  
        self.next_id = 1
```

```
    def create_order(self, items: list[str]) -> Order:  
        """Создать заказ."""  
        order = Order(self.next_id, items)  
        self.orders[self.next_id] = order  
        self.next_id += 1  
        print(f"Создан {order.info()}")  
        return order
```



```

def cancel_order(self, order_id: int) -> None:
    """Отменить заказ."""
    order = self.orders.get(order_id)
    if order is None:
        print(f"Заказ №{order_id} не найден.")
        return

    order.status = "Отменён"
    print(f"Заказ №{order_id} отменён.")

def repeat_order(self, order_id: int) -> Order | None:
    """Повторить заказ."""
    old_order = self.orders.get(order_id)
    if old_order is None:
        print(f"Заказ №{order_id} не найден.")
        return None

    return self.create_order(old_order.items)

```

```

# pylint: disable=too-few-public-methods
class Command(ABC):
    """Абстрактная команда."""

```

```

    @abstractmethod
    def execute(self):
        """Выполнить команду."""

```

```

# pylint: disable=too-few-public-methods
class CreateOrderCommand(Command):
    """Команда создания заказа."""

```

```

    def __init__(self, pizzeria: Pizzeria, items: list[str]) -> None:
        self.pizzeria = pizzeria
        self.items = items

    def execute(self) -> Order:
        return self.pizzeria.create_order(self.items)

```

```

# pylint: disable=too-few-public-methods
class CancelOrderCommand(Command):
    """Команда отмены заказа."""

```

```

    def __init__(self, pizzeria: Pizzeria, order_id: int) -> None:
        self.pizzeria = pizzeria
        self.order_id = order_id

    def execute(self) -> None:
        self.pizzeria.cancel_order(self.order_id)

```

```

# pylint: disable=too-few-public-methods
class RepeatOrderCommand(Command):
    """Команда повтора заказа."""

```

```

    def __init__(self, pizzeria: Pizzeria, order_id: int) -> None:
        self.pizzeria = pizzeria

```

```

        self.order_id = order_id

    def execute(self) -> Order | None:
        return self.pizzeria.repeat_order(self.order_id)

class OrderManager:
    """Вызывающий объект."""

    def __init__(self) -> None:
        self.history: list[str] = []

    def run(self, command: Command):
        """Запустить команду и сохранить её в историю."""
        result = command.execute()
        self.history.append(command.__class__.__name__)
        return result

def main() -> None:
    """Демонстрация работы паттерна."""
    pizzeria = Pizzeria()
    manager = OrderManager()

    first_order = manager.run(CreateOrderCommand(pizzeria, ["Пепперони", "Кола"]))
    second_order = manager.run(CreateOrderCommand(pizzeria, ["Маргарита", "Сок"]))

    manager.run(CancelOrderCommand(pizzeria, first_order.order_id))
    manager.run(RepeatOrderCommand(pizzeria, second_order.order_id))

    print("\nИстория команд:")
    for item in manager.history:
        print(item)

if __name__ == "__main__":
    main()

```

Вывод:

Пример:

```

Создан Заказ №1: Пепперони, Кола, статус=Создан
Создан Заказ №2: Маргарита, Сок, статус=Создан
Заказ №1 отменён.
Создан Заказ №3: Маргарита, Сок, статус=Создан

История команд:
CreateOrderCommand
CreateOrderCommand
CancelOrderCommand
RepeatOrderCommand

```

Вывод: В процессе выполнения работы были изучены и применены ключевые паттерны проектирования из трёх основных групп: порождающие, структурные и поведенческие. Реализация заданий позволила на практике увидеть, как паттерны помогают организовывать архитектуру программы, разделять ответственность между объектами и упрощать расширение функциональности.